UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL
INGENIERÍA EN INFORMÁTICA Y SISTEMAS

LABORATORIO DE SISTEMAS OPERATIVOS

INFORME TÉCNICO DE UNIDAD 1

"Aplicación MiniSO"

INTEGRANTES:

NINA ESTUCO, William Andre **2024-119006**
PINO ROJAS, Raul Farid Banchello **2024-119009**
ORDOÑO QUISPE, Hugo Antonio **2024-119049**

TACNA - PERÚ

2026

1. Introducción

Un sistema operativo es el software que administra los recursos de la computadora (procesador, memoria y dispositivos) y ofrece a los programas una interfaz uniforme para utilizarlos (Silberschatz et al., 2018). Tres de sus responsabilidades centrales son la administración de procesos, la coordinación de tareas concurrentes y la gestión de la memoria. Comprenderlas exige observar cómo se concretan en llamadas al sistema, estructuras de datos y algoritmos, y no solo conocer sus definiciones.

MiniSO es una aplicación de consola desarrollada en C++ para Linux que reúne, en un único ejecutable, tres módulos accesibles desde un menú principal: (1) gestión de procesos, (2) gestión de hilos y concurrencia y (3) gestión de memoria, esta última dividida en particionamiento contiguo con First-Fit y paginación. El propósito del proyecto es disponer de un entorno observable donde cada concepto de la primera unidad pueda ejecutarse, modificarse y analizarse.

Una distinción atraviesa todo el informe: los módulos de procesos e hilos operan sobre procesos e hilos reales de Linux, creados mediante llamadas al sistema y primitivas de sincronización; en cambio, el módulo de memoria es una simulación, pues los bloques, marcos y tablas de páginas son estructuras de datos del programa y no memoria real administrada por el núcleo.

Tabla 01. Taxonomía de los módulos del proyecto de Sistemas Operativos

Módulo	Mecanismo principal	Naturaleza
Procesos	fork(), execl(), waitpid()	Real (llamadas al sistema)
Hilos	std::thread, std::mutex, std::condition_variable	Real (hilos del núcleo)
Particionamiento	Lista de bloques, First-Fit, unión de libres	Simulación
Paginación	Marcos, tabla de páginas, traducción de direcciones	Simulación

1.1 Objetivos del Proyecto MiniSO

- Diseñar, implementar y evaluar una aplicación que integre los principales conceptos teóricos y prácticos estudiados durante la primera unidad de la asignatura de Sistemas Operativos, mediante la programación de procesos, hilos, mecanismos de sincronización y técnicas de gestión de memoria.
- El proyecto permitirá demostrar la comprensión del funcionamiento interno de un sistema operativo, la capacidad para resolver problemas de programación de sistemas y la aplicación de mecanismos de administración de recursos.

1.2 Relación con los Contenidos de la Unidad

- **Creación y Ciclo de Vida de Procesos:** Invocación de fork(), obtención de PIDs (getpid(), getppid()), sustitución de ejecutable (execl) y sincronización con wait().
- **Concurrencia y Multihilo:** Creación y join de hilos con pthread_create() y pthread_join().
- **Exclusión Mutua y Sincronización:** Cerrojos Mutex y señales de condición (pthread_cond_wait, pthread_cond_signal).
- **Gestión de Memoria:** Algoritmo First-Fit sobre particiones contiguas y traducción de direcciones lógicas a físicas (N_página = dir / TAM, Desplazamiento = dir % TAM).

2. Diseño de la Solución

2.1. Arquitectura General de la Aplicación

MiniSO sigue una arquitectura modular en la que un programa principal presenta un menú y delega la ejecución en funciones públicas de cada módulo. Los módulos no intercambian datos entre sí: cada uno encapsula su lógica y su estado, y solo se comunica con el exterior a través de una función declarada en su archivo de cabecera.

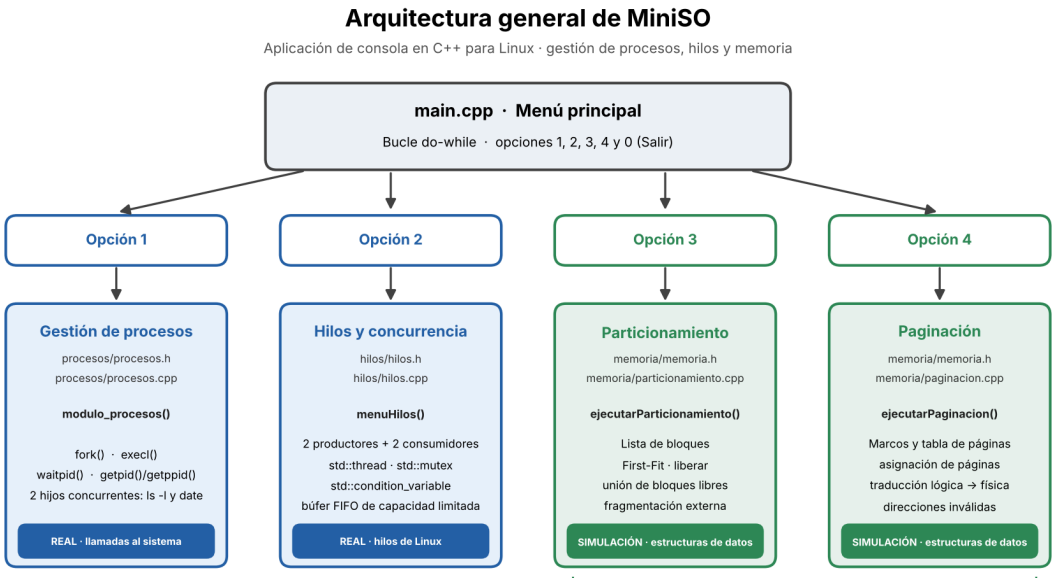


Figura 01. Arquitectura del proyecto MiniSO.

Tabla 02. Organización modular y estructura de archivos

Archivo o directorio	Responsabilidad
main.cpp	Menú principal e integración de los módulos
procesos/procesos.h, procesos.cpp	Módulo I. Función pública modulo_procesos()
hilos/hilos.h, hilos.cpp	Módulo II. Función pública menuHilos()
memoria/memoria.h	Declara ejecutarParticionamiento() y ejecutarPaginacion()
memoria/particionamiento.cpp	Módulo III-A: particionamiento contiguo First-Fit
memoria/paginacion.cpp	Módulo III-B: paginación y traducción de direcciones
Makefile	Compilación de los cinco archivos fuente y generación del ejecutable miniso
Readme.md	Documentación del repositorio
docs/pruebas/	Capturas de pantalla de las pruebas

2.2. Funcionamiento del menú principal e integración

El menú de main.cpp ofrece cinco opciones: procesos (1), hilos (2), particionamiento (3), paginación (4) y salida (0). La opción elegida invoca la función del módulo correspondiente, que retoma el control del flujo hasta que el usuario regresa al menú. El módulo de memoria se expone en dos opciones porque sus dos mecanismos tienen submenús independientes. El estado de la simulación de memoria se mantiene en variables static de cada archivo, por lo que persiste mientras el programa permanece abierto, aun si el usuario sale y vuelve a entrar al submenú.

2.3. Herramientas y tecnologías

Se utilizó C++ con el compilador g++ (opciones -Wall -std=c++11), GNU Make, las interfaces POSIX unistd.h y sys/wait.h para procesos, y los componentes std::thread, std::mutex y std::condition_variable de la biblioteca estándar, que en Linux se apoyan en POSIX Threads. No se emplean bibliotecas externas. Las pruebas del equipo se ejecutaron en Ubuntu sobre una máquina virtual VirtualBox, según las capturas disponibles. Para compilar y ejecutar: make y luego ./miniso. El Makefile no incluye -pthread; en la verificación realizada (Ubuntu 24.04, g++ 13.3) la compilación y el enlace fueron correctos sin esa opción, aunque se recomienda añadirla por portabilidad.

3. Implementación

Módulo I: Gestión de Procesos (procesos/procesos.cpp)

- Se implementó la bifurcación del proceso mediante fork().
- En el proceso hijo se imprimen su PID y el PPID del padre, y se ejecuta el binario del sistema /bin/ls mediante execl("/bin/ls", "ls", (char *)NULL). Se incluye control de errores en caso de fallo de execl.
- En el proceso padre se suspende la ejecución esperando al hijo mediante wait(NULL), confirmando su finalización antes de retornar al menú principal.

```
--- MODULO I: GESTION DE PROCESOS ---
[Padre] PID: 41642 | Hijos creados -> Hijo 1: 41647, Hijo 2: 41648
[Hijo 1] PID actual: 41647 | PPID (Padre): 41642
[Hijo 1] Ejecutando programa externo (/bin/ls)...
[Hijo 2] PID actual: 41648 | PPID (Padre): 41642
[Hijo 2] Ejecutando comando externo (/bin/date)...
[Padre] Esperando la finalizacion de ambos procesos hijos de forma concurrente...
total 268
-rw-rw-r-- 1 raul raul    710 Nov  3 23:41 Makefile
-rw-rw-r-- 1 raul raul    723 Nov  4 00:07 Readme.md
drwxrwxr-x 3 raul raul   4096 Nov  3 23:28 docs
drwxrwxr-x 2 raul raul   4096 Nov  4 00:08 hilos
-rw-rw-r-- 1 raul raul   1257 Nov  4 00:07 main.cpp
-rw-rw-r-- 1 raul raul   3768 Nov  4 00:07 main.o
drwxrwxr-x 2 raul raul   4096 Nov  4 00:07 memoria
-rwxrwxr-x 1 raul raul 238464 Nov  4 00:08 miniso
drwxrwxr-x 2 raul raul   4096 Nov  4 00:08 procesos
[Padre] El proceso hijo (PID: 41647) termino normalmente con codigo de salida: 0
mié 04 nov 2026 00:08:51 -05
[Padre] El proceso hijo (PID: 41648) termino normalmente con codigo de salida: 0
```

Figura 02. Ejecucion de Procesos.

Módulo II: Gestión de Hilos y Concurrencia (hilos/hilos.cpp)

Se resuelve el problema del Productor-Consumidor con búfer acotado (tamaño 5):

- Uso de pthread_mutex_t mutex para garantizar exclusión mutua sobre la sección crítica.
- Uso de pthread_cond_t cond_espacio y pthread_cond_t cond_elemento para coordinar el estado del búfer lleno o vacío.
- Creación de hilos productores y consumidores sincronizados que producen y consumen elementos sin generar pérdidas ni interbloqueos.

```
====== GESTION DE HILOS Y CONCURRENCIA ======
1. Simulacion personalizada (elementos, capacidad y velocidad)
2. Prueba A: capacidad 3, 20 elementos
3. Prueba B: capacidad 5, 30 elementos (consumidores lentos)
4. Prueba C: capacidad 2, 10 elementos (productores lentos)
5. Ejecutar las tres pruebas (A, B y C)
0. Volver al menu principal
```

Figura 03. Menú de Hilos.

Módulo III: Gestión de Memoria (memoria/particionamiento.cpp y memoria/paginacion.cpp)

- **Particionamiento Contiguo (First-Fit):** Algoritmo que busca el primer bloque libre con tamaño suficiente para alojar el proceso solicitado,

registrando el estado de bloques ocupados y libres para evaluar la fragmentación externa.

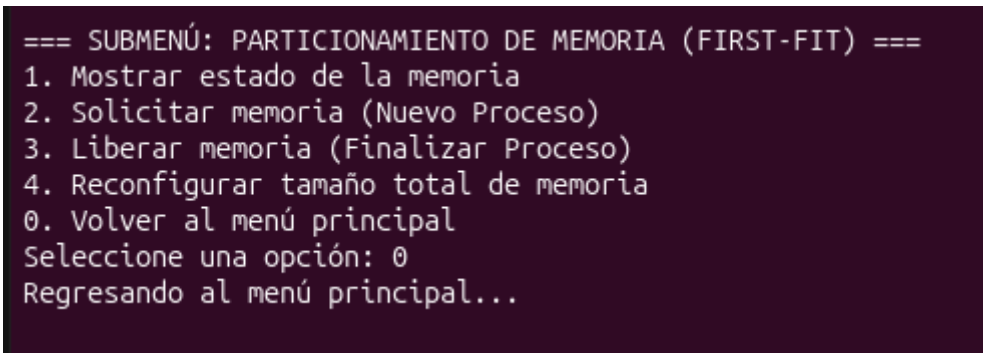


Figura 04. Menú de Particionamiento.

- **Paginación y Traducción de Direcciones:** Cálculo del número de página y desplazamiento a partir de una dirección lógica ingresada por el usuario. Mapeo a marcos de memoria física mediante la tabla de páginas para determinar la dirección física final.

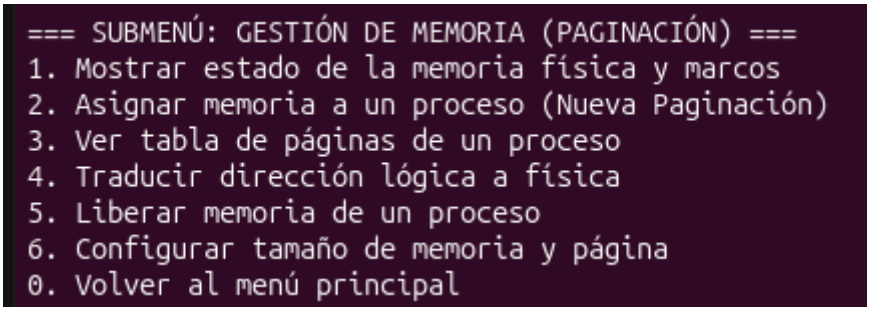


Figura 05. Menú de Paginación.

4. Pruebas planteadas

Tabla 03. Pruebas planteadas

Módulo	**Entrada / Parámetros**	**Resultado Esperado**	**Mecanismo de Verificación**
Módulo I: Procesos	Ejecución de ejecutar_modulo_procesos()	Imprime PID/PPID, ejecuta ls y el padre espera la terminación del hijo	Verificación visual y contraste de PID con ps / pstree en Linux

Módulo	Entrada / Parámetros	Resultado Esperado	Mecanismo de Verificación
Módulo II: Hilos	Productor-Consumidor con búfer = 5	Mensajes intercalados de producción y consumo con espera pasiva según estado	Inspección del orden de eventos sin inconsistencias
Módulo III: First-Fit	Solicitud y liberación de bloques	Asignación en la primera partición disponible adecuada	Inspección del estado de memoria libre/ocupada
Módulo III: Paginación	Dirección lógica ingresada	Cálculo exacto de página, desplazamiento y dirección física	Comprobación aritmética manual

Los resultados se encuentra en docs/pruebas

5. Dificultades Encontradas y Soluciones

- **Evitar despertares espurios e inanición en hilos:**
  - *Solución:* Implementar verificaciones continuas (while) en torno a pthread_cond_wait para garantizar que la condición de búfer se cumpla antes de modificar variables compartidas.
- **Modularidad en C++ con Makefile:**
  - *Solución:* Separación clara de archivos de cabecera (.h) y código fuente (.cpp) por carpetas (procesos, hilos, memoria) e inclusión de la bandera -pthread en el Makefile.

6. Conclusiones

- Se diseñó, implementó y evaluó con éxito el simulador **MiniSO**, logrando consolidar de forma práctica los conceptos teóricos fundamentales de los sistemas operativos. A través de una arquitectura modular en C++, se integraron de manera funcional los módulos de gestión de procesos (`fork` y control de PID/PPID), hilos concurrentes, búferes compartidos y técnicas avanzadas de administración de memoria, abarcando tanto el particionamiento dinámico (First-Fit) como la paginación virtual con traducción de direcciones y control de límites.
- El desarrollo del proyecto demostró una comprensión profunda del funcionamiento interno de un sistema operativo y de la gestión a bajo nivel en entornos Linux. Asimismo, se evidenció la capacidad técnica para resolver problemas complejos de programación de sistemas mediante la implementación de algoritmos de asignación de recursos, control de concurrencia y validación de estados excepcionales (como violaciones de segmento y gestión de memoria insuficiente).

7. Referencias Bibliográficas

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
- Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.
- Kerrisk, M. (2010). *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10.ª ed.). Wiley.